

Inputs, outputs, and console behaviour for each of the five scripts

What to pass, what each produces and where, the columns in every output, and how to read the console in the success, edge-warning and error cases.

Assets DC→Cloud migration · attribute remediation series · document 3 of 4

Document set: (1) Data Model & Concepts · (2) Process & Stages · (3) Scripts Reference · (4) Runbook, Recovery & Rollback. Each part is standalone and can be regenerated independently.

1. Common conventions

- **Language / runtime.** Windows PowerShell 5.1 compatible. TLS 1.2 is forced for HTTPS to older DC.
- **DC auth.** A Personal Access Token (PAT), sent as `-Pat` (Bearer). Without it the scripts fall back to `-UseDefaultCredentials` (Kerberos/NTLM on a domain machine).
- **Cloud auth.** Atlassian account email + API token, used as HTTP Basic. Token from id.atlassian.com/manage/api-tokens.
- **Dry-run.** The two writing scripts (3, 4) do nothing destructive unless `-Execute` is given. Without it they print the intended request and stop.
- **Encoding.** All REST values are UTF-8 URL-encoded; Confluence titles with German umlauts / ß round-trip correctly.
- **Pagination.** AQL queries page automatically until exhausted.

Golden rule. Scripts 3 and 4 mutate data. Run them with no `-Execute` first, read the printed request, confirm the backup files were written, then re-run with `-Execute` — on the sandbox before anything else.

2. Get-AssetsMigrationBlockers-Audit.ps1 (Script 5, run first)

Audits one or more schemas for all blocker attribute types and the inheritance / AQL-usage relationships around them. Run before any remediation.

Key inputs

Parameter	Meaning
<code>-BaseUrl / -Pat</code>	DC base URL and token.
<code>-SchemaId</code>	One id or comma-separated list.
<code>-BlockerTypes</code>	Type ids treated as blockers; default 3,5,6 (Confluence, Version, Project).
<code>-CountPopulated</code>	Also count populated objects per blocker (extra AQL calls).

Output

Two CSVs. `assets_attribute_audit.csv`: one row per attribute across every object type, with TypeLabel, IsBlocker, IsInherited, InheritedOnObjectTypes, RefObjectTypeId and a config snippet. `assets_blocker_findings.csv`: one row per blocker with DeclaredOnTypes, IsInherited, PopulatedObjects and ReferencedByAql.

How to read the console

Case	What you see
success	A type breakdown table, then a BLOCKERS section listing each blocker, the object types it sits on, and (green) <i>None found</i> if the schema is clean.
edge / warning	Yellow <i>INHERITED across multiple object types</i> notes, and a closing reminder to cross-check non-default types against the current JCMA matrix.
issue	Red <i>AQL USAGE: referenced by ...</i> lines — a reference filter depends on the blocker; do not remove the attribute before fixing the filter. HTTP errors throw with the failing URL.

3. Get-AssetsConfluenceLinks-Batch-v2.ps1 (Script 1, inventory)

Reads an input CSV of (SchemaId, AttributeName, AttributeType), keeps the Confluence rows, and lists every populated object with its value, parsed link parts and a raw-value backup.

Key inputs

Parameter	Meaning
<code>-InputCsv</code>	CSV with headers SchemaId, AttributeName, AttributeType.
<code>-BaseUrl</code> / <code>-Pat</code>	DC base URL and token.
<code>-TypeFilter</code>	AttributeType value kept; default Confluence.
<code>-OutCsv</code> / <code>-CoverageCsv</code> / <code>-BackupDir</code>	Output locations.

Output

`confluence_attr_values.csv` (one row per value, with FlatValue, LINK_URL, SpaceName, DC_ConfPageId, RawValueJson, plus blank Action/Comments), `confluence_coverage.csv` (populated counts per object type), and `rawvalue_backup\<Key>_Attribute<AttrId>_RawValue.txt` per object.

How to read the console

Case	What you see
success	Per attribute: <i>declared on N object type(s); populated objects: M</i> , then a coverage table and the three output paths.
edge / warning	Amber NOTE lines when an attribute resolves to Jira type 0 (a plain link stored as text) rather than 3 — these are still extracted, not dropped. A blank DC_ConfPageId means the page id was not present in that value and needs the inspect step.
issue	<i>Attribute not found on any object type</i> warnings (input name mismatch) and thrown HTTP errors with the URL.

4. ConfluenceDcToCloud-PageMap.ps1 (Script 2, mapping)

Maps each DC Confluence link to its Cloud equivalent by space + title via CQL. Handles pretty (/display/SPACE/Title) and pageld URLs; resolves missing titles from DC Confluence when configured.

Key inputs

Parameter	Meaning
<code>-InputCsv</code>	CSV with LINK_URL, SpaceName, DC_ConfPageld.
<code>-CloudBaseUrl</code> / <code>-CloudEmail</code> / <code>-CloudToken</code>	Confluence Cloud base (incl. /wiki) and Basic-auth credentials.
<code>-DcConfluenceBaseUrl</code> / <code>-DcConfluencePat</code>	Optional, to resolve title from a pageld-only link.
<code>-OutCsv</code> / <code>-DelayMs</code>	Output and optional throttle.

Output

`confluence_pagemap.csv` : LINK_URL, SpaceName, DC_ConfPageld, Cloud_URL, Cloud_ConfPageld, plus Title and MatchStatus for triage.

How to read the console

Case	What you see
success	A per-row progress line and a final <i>Mapped X/Y pages</i> . MatchStatus <code>OK_EXACT</code> is the clean case.
edge / warning	MatchStatus <code>OK_FUZZY_EXACT</code> (matched via a fallback search), <code>MULTIPLE_TOOK_FIRST</code> (several pages share the title — verify), or <code>NO_TITLE</code> (no title and no DC lookup available).
issue	<code>NOT_FOUND</code> (no Cloud page by that space+title — check space-key remap or title drift) or <code>ERROR: ...</code> with the message. Review every non-OK row before relying on the map.

5. Clear-AssetsAttribute-DC.ps1 (Script 3, pre-migration write)

Clears one attribute's value on one DC object (keeping the attribute on the schema) so it stops blocking JCMA. Dry-run by default.

Key inputs

Parameter	Meaning
<code>-Object</code>	Object id or key (e.g. CMDB-3493).
<code>-AttrId / -AttributeName</code>	Target attribute (one of the two).
<code>-Execute</code>	Actually perform the clear; omit for dry-run.
<code>-BackupDir</code>	Where before/after JSON backups go.

Output

Before/after full-object JSON backups, and on `-Execute` the value set to []. Files: `<Key>_BEFORE_object/attributes.json.txt` and (executed) `<Key>_AFTER_*`.

How to read the console

Case	What you see
success	A PLAN block (object, attribute, current value, exact PUT body) then, in dry-run, <i>DRY RUN — no change made</i> with two web-view links to verify after executing.
edge / warning	If the object key is ambiguous or the attribute name resolves to more than one definition, it stops with a clear message rather than guessing.
issue	Thrown errors with the failing URL. After execute, the printed <i>Value now</i> and the AFTER backup let you confirm only the target attribute changed.

6. New-And-Set-AssetsUrlAttribute-Cloud.ps1 (Script 4, post-migration write)

In Cloud Assets: auto-discovers the workspace, creates a replacement URL/Text attribute on the object type, then sets its value on one object to the mapped Cloud URL. Staged, dry-run by default.

Key inputs

Parameter	Meaning
<code>-CloudBaseUrl</code> / <code>-CloudEmail</code> / <code>-CloudToken</code>	Jira Cloud base and Basic-auth credentials.
<code>-ObjectTypeId</code> / <code>-ObjectId</code>	Where to create the attribute, and which object to set.
<code>-NewAttrName</code> / <code>-NewAttrType</code>	Replacement name and URL/Text/Textarea.
<code>-Value</code>	The Cloud URL (from the page map).
<code>-ExistingAttrId</code>	Reuse an already-created attribute (skips creation).
<code>-Execute</code>	Apply all stages; omit for dry-run.

Output

Before/after backups of the object type attributes and the object:

`Cloud_ObjectType<id>_BEFORE/AFTER_attributes.json.txt` and `Cloud_Object<id>_BEFORE/AFTER.json.txt`.

How to read the console

Case	What you see
success	Four stage banners (discover → backup → create → set), each saying what it will do and how to verify in the web view; dry-run shows the create and set bodies with a placeholder attribute id.
edge / warning	An amber NOTE if an attribute of that name already exists (suggesting <code>-ExistingAttrId</code> to avoid a duplicate).
issue	Thrown errors with the failing URL — e.g. if the create body field name differs on your Cloud version, the create call reports it and you adjust the key.